

Coarse-to-fine sampling: target, generator, sampler

Hierwave notes

Three objects, kept separate throughout:

1. the **target**, an undirected energy model on tiles that says what we want to sample;
2. the **generator**, a chain graph over coarse-to-fine latent variables that approximates it;
3. the **sampler**, the prolong–perturb–smooth pass that approximately samples the generator.

Code: `castlegen/hier.py`, `texsyn.py` (CoordVar), `promise.py`, `quantities/`. Companion to `texture_sampling.md`, `promises.md`, `promise_estimation.md`.

1 The target: an undirected model on tiles

The tile set defines a Boltzmann distribution on tile grids x ,

$$p(x) \propto e^{-E_p(x)/T}, \quad E_p = \sum \text{pair tables} + \sum \text{unaries},$$

and a guarantee G (connectivity, flow, support) restricts it to $p_G(x) \propto p(x) \mathbb{1}[G(x)]$.

Promises are deterministic functions of the tiles. For every block b of side $h \geq K$ in a fixed quadtree, $\pi_b = A_b(x)$, with an exact merge $\pi_{\text{parent}} = \text{merge}(\pi_{\text{children}})$. Adding them to the target adds only hard, undirected factors:

$$p_G(x, \pi) \propto p(x) \mathbb{1}[G(x)] \prod_b \mathbb{1}[\pi_b = A_b(x)],$$

whose x -marginal is still p_G . *Specifying* promise values π^* on a set of blocks B is ordinary conditioning, $p_G(x \mid \pi_B = \pi^*)$. The induced distribution of promise pyramids is the push-forward $q = A_{\#}p_G$.

The target has no levels and no direction; everything below is about sampling it with local operations.

2 The generator: a chain graph

Levels are indexed by cell side $h \in \{m, m/2, \dots, 1\}$ (m the exemplar side). Level h is one *chain component* $C_h = \{S_h, \pi_h\}$:

- $S_h : \mathbb{Z}_{n/h}^2 \rightarrow \mathbb{Z}_m^2$, exemplar coordinates (an auxiliary latent);
- π_h , the promise grid, present only for $K \leq h \leq h_{\text{top}}$.

The tiles form a last component. The generator is a product of *locally normalised* level conditionals, directed coarse to fine:

$$g(S, \pi, x) = g_{\text{top}}(C_{\text{top}}) \prod_h g_h(C_h \mid C_{2h}) g(x \mid S_1, \pi_K), \quad (1)$$

$$g_h(C_h \mid C_{2h}) = \frac{1}{Z_h(C_{2h})} \underbrace{\psi_h(S_h, S_{2h}) \psi_h^\pi(\pi_h, \pi_{2h})}_{\text{directed, between levels}} \underbrace{\varphi_h(S_h) \varphi_h^\pi(\pi_h) \chi_h(S_h, \pi_h)}_{\text{undirected, within level}}, \quad (2)$$

$$g(x \mid S_1, \pi_K) \propto \tau(x, S_1) e^{-E_p(x)/T} \prod_{b: h_b=K} \mathbb{1}[A_b(x) = \pi_b]. \quad (3)$$

Factor	Form	Code
ψ_h	$k_h(S_h - US_{2h})$, prolongation U plus a noise kernel	<code>CoordVar.init_children</code>
φ_h	$e^{-\mathcal{E}_h(S_h)/T}$, patch energy against the exemplar	<code>CoordVar.energy</code>
ψ_h^π	$\mathbb{1}[\text{merge}(\pi_h) = \pi_{2h}] e^{-c_h(\pi_h, \pi_{2h})}$	<code>refine, Tables.parent</code>
φ_h^π	$\mathbb{1}[\text{consistent}(\pi_h)] e^{-\sum u_h - \sum e_h}$	<code>consistent, Tables.u/pair</code>
χ_h	$e^{-\lambda \text{cost}_h(S_h, \pi_h)}$, texture honours promise	<code>CoordPromiseCoupling</code>
top	$\varphi_{\text{top}} \varphi_{\text{top}}^\pi \mathbb{1}[G(\pi_{\text{top}})] \chi_{\text{top}}$	<code>top</code>
τ	coupling of tiles to texture, $e^{-w d(x, E[S_1])}$	<code>heights stage</code>

Why a chain graph and not one undirected energy. With a single global normaliser the fine levels would feed back on the coarse ones: the effective coarse distribution becomes $\varphi_{2h} Z_h(C_{2h})$, tilted by how many fine configurations each coarse value admits. Local normalisation buys three things the design depends on:

1. **Locality.** A window of level h depends only on a padded window of level $2h$, so chunked, infinite generation is well defined.
2. **Meaningful tables.** A level's factors can be fitted as a conditional given the parent (counts now, pseudo-likelihood in `plfit.py`), and mean what they were fitted to mean.
3. **Ancestral sampling.** Sampling C_{top} , then each C_h given C_{2h} , is exact for the generator (Section 4).

Promises point both ways. In the target, promises are defined bottom-up ($x \rightarrow \pi_K \rightarrow \pi_{2K} \rightarrow \dots$); in the generator they are sampled top-down. The two directions cannot share one chain graph: with χ_h linking S_h and π_h , $S_h \rightarrow \dots \rightarrow S_1 \rightarrow x \rightarrow \pi_K \rightarrow \dots \rightarrow \pi_h - S_h$ would be a partially directed cycle. The bottom-up definition is therefore not an edge of the generator but a condition on it (Section 3).

3 When the generator equals the target

Let $\hat{q}$ be the generator's marginal over π and $g(x \mid \pi)$ its conditional over tiles with S marginalised out. Then

$$\begin{aligned} \hat{q} &= q = A_{\#} p_G \quad \text{and} \quad g(x \mid \pi) = p_G(x \mid A(x) = \pi) \\ \implies g(x) &= \sum_{\pi} q(\pi) p_G(x \mid A(x) = \pi) = p_G(x). \end{aligned}$$

Neither condition is needed for G : every consistent π with $G(\pi_{\text{top}})$ implies G for any x that honours it (the soundness theorem of each promise language). The conditions measure how *typical* the output is, not whether it is valid.

Three known departures:

- φ_h encodes the exemplar’s statistics, which match p locally but not at macro scale. S is there to cross barriers local Gibbs on p cannot; the relaxation on E_p afterwards pulls x back toward p .
- $\hat{q}$ is fitted to a corpus. It approaches q only if the corpus is drawn from p_G (e.g. by an exact reference sampler, not as repaired samples $R_{\#}p$) and the fitted model generalises to the configurations the sampler visits.
- A contract weaker than $A_b(x) = \pi_b$ (e.g. promises as lower bounds) preserves G but not the second condition. The languages of Section 5 use equality.

How far off, exactly. Suppose the tile level samples its conditional exactly, $g(x \mid \pi_K) = p_G(x \mid A_K(x) = \pi_K)$. Then $g(x) = \sum_{\pi} \hat{q}(\pi) p_G(x \mid \pi)$, and since $\pi_K = A_K(x)$ is a function of x ,

$$\text{TV}(g, p_G) = \text{TV}(\hat{q}_K, q_K).$$

Every error of the fitted promise distribution at level K passes to the output, and nothing else does. More sweeps do not reduce it: sweeps equilibrate a level given the level above, and nothing below ever pushes back on a coarser level.

Conditioning. Clamping $\pi_B = \pi^*$ in the generator reproduces the target’s conditional only if $\hat{q}(\pi \mid \pi_B = \pi^*) = q(\pi \mid \pi_B = \pi^*)$. Upward this is deterministic (merge fixes the ancestors that B fully covers). Downward, a top-down sampler must avoid coarse values with no consistent completion that reaches π^* , which needs a backward feasibility message

$$\beta_h(\pi_h) = \mathbf{1}[\pi_h \text{ has a consistent refinement compatible with } \pi^*],$$

as in forward-filtering backward-sampling.

4 The sampler: prolong, perturb, smooth

Each level conditional $g_h(C_h \mid C_{2h}) \propto \psi \varphi$ is sampled by splitting it into its two kinds of factor:

Step	Operation	Role
Prolong	$\hat{C}_h = \text{location of } \psi_h, \psi_h^\pi \text{ given } C_{2h}$	mode of the directed factor
Perturb	$C_h^{(0)} \sim \psi_h(\cdot, C_{2h})$	exact draw from the directed factor
Smooth	$C_h = R_h^{n_h}(C_h^{(0)})$	short-run MCMC/ICM on the within-level factors

Prolong + perturb is ancestral sampling of the directed part; smoothing is truncated (block) Gibbs on the undirected part. The full pass,

$$C_1 = R_1^{n_1} \circ K_1 \circ R_2^{n_2} \circ K_2 \circ \dots \circ R_{\text{top}}(\emptyset), \quad K_h = \text{perturb} \circ \text{prolong},$$

is cascadic multigrid (prolong and smooth, never restrict), or multiscale ancestral sampling with short-run MCMC correction. The smoother is exact for g_h only if its target keeps the directed factor; otherwise the parent survives only through the initialisation and the small n_h .

Variable	Prolong	Perturb	Smooth
S_h	US_{2h}	jitter k_h	ICM/Gibbs on $\varphi_h \chi_h$ (ψ_h dropped)
π_h	MAP over F (Sec. 5)	draw over F	exact chain-block Gibbs on $\psi_h^\pi \varphi_h^\pi \chi_h$ (ψ_h^π kept)
x	feasible x nearest $E[S_1]$	none	MCMC on $\tau e^{-E_p/T}$ that keeps $A_b(x) = \pi_b$ exactly

5 Promise machinery

This section states what a promise language must provide and how the sampler uses it, for any guarantee G . Section 6 works it through for support.

5.1 What a language provides

A block b of side h has an exact state s_b , computed from x_b , with an exact merge $s_{\text{parent}} = M(s_{\text{children}})$. A promise language supplies:

- an **abstraction** A from states to values, $\pi_b = A(s_b)$, depending only on the block’s own contents, so the language is translation invariant;
- a **promise merge** m that commutes with it: $A(M(s_1, \dots, s_4)) = m(A(s_1), \dots, A(s_4))$;
- **validity** $\mathcal{V}_{\text{pos}}(\pi)$: which values a block may take at its position (boundary conditions);
- a **seam relation** $\pi \sim_d \pi'$ for blocks adjacent in direction d ;
- the **contract** at level K : x honours π_K iff $G(x)$ and $A_b(x) = \pi_b$ for every K -block.

A grid of values is *consistent* if every value is valid, every seam obeys $\sim$, and every 2×2 group merges to its parent. The language must have four properties.

1. **Soundness.** A consistent pyramid whose K -blocks are honoured implies G .
2. **Completeness.** $G(x)$ implies that $A(x)$ is a consistent pyramid. Then q lives on consistent pyramids, and a corpus can be abstracted directly, with no projection.
3. **Realisability.** Every consistent value has a consistent refinement for any consistent halo, and every consistent level- K grid has a completion x . Top-down sampling then never gets stuck.
4. **Exact seam rule.** $\pi \sim_d \pi'$ iff some pair of states with those abstractions obeys the tile-level rule across the seam. A stronger rule is sound but forbids valid configurations, which shows up as systematic artifacts. A weaker rule is unsound.

5.2 Feasible sets and chain blocks

For a parent p at level $2h$, the *feasible set* is

$$F(p) = R(\pi_{2h}(p) \mid \text{halo}) \cap \beta_h \cap \text{clamps},$$

where R is the set of consistent refinements given the neighbours’ current children, and β_h is the conditioning message below. Every step for π_h (prolong, perturb and smooth) chooses within F , so every level is consistent after every step.

Seam interfaces. The seam relation reads only a projection $\iota_d(\pi)$, the *interface* a value presents across side d . A parent’s value need not fix what its children present at the parent’s external seams. When it doesn’t, refinement depends on the halo, and the level conditional couples neighbouring parents. A per-parent draw is then exact only given the halo, which is fine for smoothing but not for the first draw of a level.

Chain blocks. Choose update blocks whose internal hard constraints form a chain $c_1, \dots, c_L$ (a path of cells or groups). Given everything outside the block, its conditional has the form

$$p(c_{1:L} \mid \text{rest}) \propto \exp\left(-\frac{1}{T} \left[\sum_i e_i(c_i) + \sum_i t_i(c_i, c_{i+1}) \right]\right),$$

where e_i collects everything fixed during the draw (unary table, parent term, pair terms with the flanking blocks, χ , clamps and β as $+\infty$), and t_i holds the hard merge and seam masks plus soft vertical pair terms. It is drawn exactly by forward-filtering backward-sampling:

$$\mu_L = 0, \quad \mu_i(c) = -T \log \sum_{c'} e^{-[t_i(c, c') + e_{i+1}(c') + \mu_{i+1}(c')]/T},$$

then $c_1 \propto e^{-[e_1 + \mu_1]/T}$ and $c_{i+1} \propto e^{-[t_i(c_i, \cdot) + e_{i+1} + \mu_{i+1}]/T}$. Taking $T \rightarrow 0$ turns the sums into minima and the draw into the MAP chain. So the three sampler steps are one computation:

Prolong	MAP chain ($T = 0$) over F
Perturb	exact draw ($T > 0$) over F
Smooth	block Gibbs, redrawing chain blocks given their neighbours, alternating block families

When the transitions depend on c_i only through its interface, the messages live on interface values, not on whole values. Small interfaces are therefore a design goal for a language.

5.3 Forward-filtering backward-sampling

Problem. A chain has L sites, each taking one of V values. It has site energies $e_i(c)$ for $i = 1, \dots, L$, transition energies $t_i(c, c')$ between sites i and $i + 1$, and a temperature $T \geq 0$. A hard constraint is an energy of $+\infty$. The target distribution is

$$p(c_{1:L}) = \frac{1}{Z} \exp\left(-\frac{E(c_{1:L})}{T}\right), \quad E(c_{1:L}) = \sum_{i=1}^L e_i(c_i) + \sum_{i=1}^{L-1} t_i(c_i, c_{i+1}).$$

Define the *soft minimum* at temperature T ,

$$\text{smin}_T x = -T \log \sum_c e^{-x(c)/T}, \quad \text{smin}_0 x = \min_c x(c),$$

with smin_T of an all- $+\infty$ vector equal to $+\infty$.

Algorithm. Algorithm 1 computes, for each site, the energy of the best (or, at $T > 0$, the softened sum over all) completions of the sites after it, and then samples from the first site forward. This is the textbook algorithm with the chain’s direction reversed: the messages run from the end, and sampling runs from the start. The code (`envelope_var.ffbs`) vectorises it over many independent chains.

Algorithm 1 Forward-filtering backward-sampling on a chain (messages from the end, samples from the start).

Require: site energies $e_{1:L}$, transition energies $t_{1:L-1}$, temperature T , noise

```

1:  $\mu_L(c) \leftarrow 0$  for all  $c$  ▷ cost-to-go after the last site
2: for  $i = L - 1, \dots, 1$  do
3:    $\mu_i(c) \leftarrow \text{smin}_T[t_i(c, \cdot) + e_{i+1}(\cdot) + \mu_{i+1}(\cdot)]$  for all  $c$ 
4: end for
5:  $a_1(c) \leftarrow e_1(c) + \mu_1(c)$ 
6: if  $a_1(c) = +\infty$  for every  $c$  then
7:   raise infeasible
8: end if
9:  $c_1 \leftarrow \text{PICK}_T(a_1)$ 
10: for  $i = 2, \dots, L$  do
11:    $a_i(c) \leftarrow t_{i-1}(c_{i-1}, c) + e_i(c) + \mu_i(c)$ 
12:    $c_i \leftarrow \text{PICK}_T(a_i)$ 
13: end for
14: return  $c_{1:L}$ 

15: function  $\text{PICK}_T(a)$ 
16:   if  $T = 0$  then
17:     return  $\arg \min_c a(c)$  ▷ ties broken by noise
18:   else
19:     draw  $g(c) \sim \text{Gumbel}(0, 1)$  independently
20:     return  $\arg \min_c [a(c)/T - g(c)]$  ▷ a draw with  $p(c) \propto e^{-a(c)/T}$ 
21:   end if
22: end function

```

Why it is exact. By induction from the end of the chain,

$$e^{-\mu_i(c)/T} = \sum_{c_{i+1:L}} \exp\left(-\frac{1}{T} \left[t_i(c, c_{i+1}) + \sum_{j>i} e_j(c_j) + \sum_{j>i} t_j(c_j, c_{j+1})\right]\right),$$

the total weight of all completions after site i given $c_i = c$. So

$$p(c_1) \propto e^{-a_1(c_1)/T}, \quad p(c_i \mid c_{1:i-1}) = p(c_i \mid c_{i-1}) \propto e^{-a_i(c_i)/T},$$

and the product of these conditionals is $p(c_{1:L})$. So one pass returns an exact sample.

- At $T = 0$ the sums become minima, and the pass returns a lowest-energy chain (the MAP chain, the Viterbi algorithm).
- Every value chosen has finite a_i . Every such value has at least one finite completion, since $\mu_i < \infty$. So once a_1 has a finite entry, no dead end can occur later.

Normaliser and likelihood. The same messages give the partition function,

$$-T \log Z = \text{smin}_T a_1,$$

so the exact log-probability of an observed chain $c_{1:L}$ is

$$\log p(c_{1:L}) = -\frac{E(c_{1:L})}{T} - \log Z.$$

This is the chain-block likelihood used to fit the level factors (`envpredict.column_nll`).

Cost. Each message is a $V \times V$ reduction, so a chain costs $O(LV^2)$ time and $O(LV)$ memory. Implementations work in log space: `smin` subtracts the minimum before exponentiating, and $+\infty$ entries contribute nothing.

How the sampler uses it. A chain is one column of promise blocks at a level.

- e_i holds the unary and parent tables, the pair terms with the fixed neighbouring columns, χ , and $+\infty$ outside the feasible sets.
- t_i holds the soft vertical pair term, plus $+\infty$ for pairs the merge or seam rule forbids.
- Prolong is the $T = 0$ call, perturb a $T > 0$ call, and each smoothing step a $T > 0$ call on every chain of one parity given the other.

5.4 Conditioning: precomputing feasibility

A request is a set of clamps $\mathcal{C} = \{(h, b) \mapsto \pi_b^*\}$ at any levels $h \geq K$. Before any sampling, two structures are computed bottom-up from K to h_{top} .

- **Feasible values** $F_h(b)$: the values block b may take and still admit a consistent completion that honours every clamp below it.
- **Feasible seams** $B_h(b, b') \subseteq \sim$ for each pair of blocks adjacent across a hard seam: the value pairs that admit such completions jointly.

At level K , and at any level for a block with nothing clamped below it,

$$F_K(b) = \begin{cases} \{\pi_b^*\} & (K, b) \in \mathcal{C} \\ \{\pi : \mathcal{V}_{\text{pos}(b)}(\pi)\} & \text{otherwise,} \end{cases} \quad B_K(b, b') = \sim \cap (F_K(b) \times F_K(b')).$$

One level up, write $\text{ch}(p)$ for the children of p , and $\text{Real}_h(P, p)$ for the refinements of value P that stay inside the feasible sets:

$$\text{Real}_h(P, p) = \{c \in \prod_{i \in \text{ch}(p)} F_h(i) : m(c) = P, \text{ every internal seam pair of } c \text{ in } B_h\}.$$

Then

$$F_{2h}(p) = \{P : \text{Real}_h(P, p) \neq \emptyset\} \cap \text{clamps at } (2h, p),$$

and $(P, P') \in B_{2h}(p, p')$ iff there are $c \in \text{Real}_h(P, p)$ and $c' \in \text{Real}_h(P', p')$ whose facing children are pairwise in B_h .

Arc consistency. A value that has no partner in some neighbour's feasible set, under B , can never be completed. So each level is pruned to a fixed point, sweeping along each chain in both directions:

$$F(b) \leftarrow \{\pi \in F(b) : \exists \pi' \in F(b') \text{ with } (\pi, \pi') \in B(b, b')\} \quad \text{for every neighbour } b',$$

with B restricted to the pruned sets.

Failure. If any F becomes empty, the clamps are infeasible. This is reported before sampling starts, never discovered as a dead end in the middle of a pass.

Why seam sets are needed. Per-block sets alone are not enough. The coarse seam rule can admit two neighbouring parents whose feasible refinements disagree across their shared seam, for example when clamps fix which half of each parent holds the matter. Arc consistency on B_h catches this. It is still only pairwise, so a combination of three or more blocks could in principle be infeasible. In practice (Section 6) no dead end appeared in 1500 random clamp patterns.

What β is. F_h and B_h together are the backward message β_h of forward-filtering backward-sampling, carried across levels. But they are its 0/1 support only, not its weight. That difference shapes the distribution the pass samples, as described below.

5.5 The tile level

The tile conditional is

$$g(x \mid S_1, \pi_K) \propto \tau(x, S_1) e^{-E_p(x)/T} \mathbb{1}[G(x)] \prod_{b: h_b=K} \mathbb{1}[A_b(x) = \pi_b].$$

Split x into the part $y = \rho(x)$ that the guarantee reads (e.g. solidity) and the rest, and reduce π_K to a feasible set $Y(\pi_K)$ for y .

- **Prolong:** $y^0 = \arg \min_{y \in Y(\pi_K)} d(y, \rho(E[S_1]))$, a projection used only as the starting point.
- **Smooth:** MCMC on the conditional whose moves stay inside $Y(\pi_K)$.
 - Local moves are checked locally, since the contract is per block plus seams.
 - Local moves alone can freeze, because a block’s abstraction can pin which cells carry a property. Moves that leave A invariant but relocate that role (e.g. permutations within a block that preserve its value) make the chain irreducible on $Y(\pi_K)$.
 - The rest of x is resampled within its class.

The projection’s bias decays with sweeps, and the limit is the exact conditional. Earlier pipelines instead used the projection as the output and repaired around it.

5.6 A conditioned pass, step by step

Given clamps $\mathcal{C}$, a seed and the fitted level factors:

1. **Precompute** F_h and B_h for every promise level (previous subsection). Stop here if any set is empty.
2. **Top level** h_{top} . The chain blocks have unaries

$$e_i(c) = u_h(c) + \lambda \text{cost}_h(S_h(b_i), c) + \infty \cdot \mathbb{1}[c \notin F_h(b_i)],$$

and transitions

$$t_i(c, c') = v_h(c, c') + \infty \cdot \mathbb{1}[(c, c') \notin B_h(b_i, b_{i+1})],$$

where u_h is the unary term, v_h the soft pair term along the chain, and the cost term is χ_h .

- Draw the even blocks first, ignoring their pair terms to other blocks.
- Draw the odd blocks given both even neighbours; the soft pair terms across become unaries.
- Then run n_{top} sweeps of block Gibbs, alternating even and odd.

3. **Each level** $h = h_{\text{top}}/2, \dots, K$, given the finished level $2h$:

- Coordinates*: prolong $S_h = US_{2h}$ and perturb with k_h .
- Promises, prolong or perturb*: the same chain draw as above, over child chains, with the parent term added to the unary and the merge added to the transitions:

$$e_i(c) = u_h(c) + c_h(c, \pi_{2h}(\text{par}(b_i)), \text{pos}(b_i)) + \lambda \text{cost}_h(S_h(b_i), c) + \infty \cdot \mathbb{1}[c \notin F_h(b_i)],$$

$$t_i(c, c') = v_h(c, c') + \infty \cdot \mathbb{1}[\text{pair not allowed}],$$

where $\text{par}(b)$ is the parent of block b , $\text{pos}(b)$ its position within that parent, and the parent term c_h comes from ψ_h^π . A pair is allowed when:

- for siblings under one parent, it is a realising pair of that parent's value, $(c, c') \in \text{Real}_h(\pi_{2h}(\text{par}), \cdot)$ restricted to the chain;
- for a pair across two parents, $(c, c') \in B_h$.

$T = 0$ gives the MAP chain (prolong); $T > 0$ gives an exact draw (perturb).

- Smooth*: n_h sweeps. Each sweep redraws the even chains given the odd ones and then the odd given the even, each an exact block-Gibbs step. It then runs the coordinate smoother on $\varphi_h \chi_h$ against the current promises.

- Below K** : coordinates only. χ couples each cell to the promise of its level- K ancestor block.
- Tile level**: reduce π_K to $Y(\pi_K)$, project $E[S_1]$ onto it, run the constrained MCMC, then resample materials within their classes.

Invariants. After every step of step 3, level h :

- is consistent with itself and with level $2h$;
- equals the clamps at level h ;
- has every value inside F_h , so a consistent completion honouring every finer clamp exists, up to the pairwise caveat above.

The tile level honours π_K exactly, and merges are exact. So every clamp, at every level, holds in the output x , together with $G(x)$.

Which distribution this samples. Let every chain draw be exact, with infinitely many sweeps. Then the pass samples the generator with each level conditional renormalised over its feasible set:

$$\tilde{g}(\pi) = g_{\text{top}}^F(\pi_{\text{top}}) \prod_h \frac{g_h(\pi_h \mid \pi_{2h}) \mathbb{1}[\pi_h \in F_h]}{Z_h^F(\pi_{2h})}, \quad Z_h^F(\pi_{2h}) = \sum_{\pi_h \in F_h} g_h(\pi_h \mid \pi_{2h}).$$

The exact conditional $g(\pi \mid \mathcal{C})$ instead weights each coarse value by the probability mass of its completions that reach the clamps. That is the soft backward message

$$\beta_h^{\text{soft}}(\pi_h) = \Pr_g(\text{finer levels honour } \mathcal{C} \mid \pi_h),$$

whose support is F_h . The two agree when β^{soft} is constant on its support. Otherwise the coarse levels of $\tilde{g}$ over-weight values with few feasible completions. The soft message is a sum over all finer pyramids, which is intractable in two dimensions. Along a single chain it is a forward pass, so chain-structured approximations are possible. This is a further departure to add to those in Section 3: with clamps,

$$\text{TV}(g(\cdot | \mathcal{C}), p_G(\cdot | \mathcal{C}))$$

depends both on $\hat{q}$ and on how far $\tilde{g}$ is from $g(\cdot | \mathcal{C})$.

5.7 Fitting the level factors

A table indexes whole values, so a value never seen whole in the corpus gets an uninformed energy. A linear predictor over language features generalises from the parts of a value that were seen:

$$E_\theta(\pi_h | \text{context}) = \theta_h \cdot \phi(\pi_h, \text{context}),$$

with ϕ built from the value’s components, its interfaces and its relation to the parent. Tables are the case where ϕ is one-hot on whole values. Fit θ_h by *chain-block pseudo-likelihood*, maximising $\sum_{\text{blocks}} \log p_\theta(c_B | \text{rest})$ over the corpus. Each term is exact by the forward pass above, and it is the conditional the smoother draws from.

Two requirements come from the data rather than the model:

- **Coverage.** The corpus must show the configurations the sampler will visit, relative to the block grid: surfaces at every offset within a block, structures crossing seams, and so on. Outside its data, a model extrapolates by its inductive bias, which can be confidently wrong.
- **Agreement of inputs.** The corpus (which sets $\hat{q}$), the exemplar (which sets χ and τ) and the target p (the tile level) must describe the same world. Where they disagree, the terms fight, and the stronger one wins locally.

6 Example: support as local height envelopes

Code: `quantities/envelope.py` (language), `envelope_var.py` (promise levels), `heights.py` (tile level), `refheights.py` (exact reference sampler), `envpredict.py` (linear predictor); experiments in `notes/experiments/`.

Guarantee. Side view with gravity. The bottom G rows are a ground band. A solid cell outside the band needs a solid or band cell directly below. Walking down from any column’s topmost solid shows that the valid maps are exactly the heightmaps: column c is air above a height $H(c) \geq G$ and solid below it.

State. For a $k \times k$ block: the local height $t(c) \in [0, k]$ of each column (the solid run from the block’s bottom row, so $t = \text{clip}(H - \text{offset}, 0, k)$), the top-row solid mask, and a count of violations. With upper child u over lower child l , the merge is

$$t = \begin{cases} t_l & t_l < k \\ k + t_u & t_l = k \end{cases}, \quad \text{viol} = \sum \text{viol} + \#\{t_u > 0 \wedge \neg \text{top}_l\}.$$

Values. Each block side is split into I column intervals. For each interval the value records the bins of $\min t$ and $\max t$, where

$$b(t) = 0 \text{ if } t = 0, \quad b(t) = F = Q + 1 \text{ if } t = k, \quad b(t) = \lceil tQ/k \rceil \text{ otherwise.}$$

With $C = Q + 2$ bins there are $C(C + 1)/2$ ordered pairs per interval and $V = (C(C + 1)/2)^I$ values. $I = Q = 2$ gives $V = 100$.

Merge. Stacked children map to the parent’s coarser bins by $\text{up}(0) = 0$, $\text{up}(b) = \lceil b/2 \rceil$ for interior b , $\text{up}(F) = Q/2$, and $\text{up}^+(b) = Q/2 + \text{up}(b)$ with $\text{up}^+(F) = F$ for the upper child:

$$\text{lo} = \begin{cases} \text{up}(l.\text{lo}) & l.\text{lo} < F \\ \text{up}^+(u.\text{lo}) & \text{else} \end{cases}, \quad \text{hi} = \begin{cases} \text{up}^+(u.\text{hi}) & u.\text{hi} > 0 \\ \text{up}(l.\text{hi}) & \text{else} \end{cases}.$$

Side by side, lo is the minimum and hi the maximum of the two child intervals a parent interval covers. The merge is exact because a column that is not full in the lower child must be empty in the upper one. Nesting bins with Q even makes the bin arithmetic exact as well.

Seam rule, validity, interfaces. For an upper value over a lower one, per interval:

$$u.\text{hi} > 0 \Rightarrow l.\text{hi} = F, \quad u.\text{lo} > 0 \Rightarrow l.\text{lo} = F.$$

If any upper column has matter at its bottom row, some lower column is full; if all do, all are. This is exactly the image of the per-column rule. The rule it replaced, “any matter above $\Rightarrow$ all columns full”, was stronger, and it produced slab artifacts.

- Side-by-side seams carry no hard constraint. The torus wrap seam, from the band up to the top row, is exempt.
- Validity: a ground-row block has $\text{lo} \geq b(G)$.
- Interfaces: an upper value presents $(\text{lo} > 0, \text{hi} > 0)$ and a lower value presents $(\text{lo} = F, \text{hi} = F)$ per interval, a few bits in all.

Properties. Completeness holds because every heightmap abstracts to a consistent pyramid. Realisability follows from clipping being monotone: in any stack of blocks, every block’s lo is attained at the interval’s lowest column and every hi at its highest. A stack is therefore consistent iff it is the clip of a common $[H_{\min}, H_{\max}]$. Given intervals at least two columns wide, any consistent level- K grid has a completion.

Chain blocks. All hard constraints run vertically, and a parent’s value over child column d depends only on that child column. So a level splits into independent *child columns*: chains of n/h values with the merge mask inside each parent and the seam rule between parents. Each column is drawn exactly by forward-filtering backward-sampling with $V = 100$ states. Horizontal pair terms become unaries when even and odd columns alternate. The per-parent β was observed to fail (a feasible-looking coarse seam forced an overhang one level down); the seam relations B_h fixed it, with no dead ends in 1500 random clamp patterns.

A conditioned pass, worked. Take $n = 128$, $K = 16$, $h_{\text{top}} = 64$, $G = 8$, $I = Q = 2$. Clamp one $h = 32$ block, at block row 1 and block column 1 (rows 32–63, columns 32–63), to *full*, $\text{lo} = \text{hi} = F$ in both intervals: a mountain.

- **Precomputation, level 16.** Nothing is clamped, so each $F_{16}(b)$ is the valid values; ground-row blocks need $\text{lo} \geq b(8) = 1$.
- **Level 32.** The clamped block’s set is $\{(F, F), (F, F)\}$. Its realising children are all full. The seam rule then forces the blocks below it in the same column to be full: block rows 2 and 3 at $h = 32$, which becomes columns 32–63 full up to row 64 at $h = 16$. So the F and B sets of that column shrink to “full” below the clamp.
- **Level 64.** The clamp is the lower-right child of parent $(0, 0)$. The lower child in that child column is full, so every column there has local height $32 + t_u \geq 32$. The parent’s interval over those columns therefore has $\text{lo}, \text{hi} \in \{1, 2, F\}$ (bin 1 is exactly height 32). By the seam rule, ground block $(1, 0)$ below it must be full over those columns.
- **Sampling.** At $h = 64$ the chain for parent column 0 draws values from these shrunken sets. At $h = 32$ the clamped block is fixed, and its column’s blocks below are forced full. The other columns, and the upper-left child of $(0, 0)$, are free, subject to the merge with their parent. The same holds down to $h = 16$.
- **Tile level.** For columns 32–63 the interval constraints give $H \geq 96$. Every other interval is constrained only by its sampled envelope.

The output always honours the clamp. How the free neighbours are distributed depends on $\hat{q}$ and on the hard-versus-soft β effect: in the experiments, count tables fitted to a flat corpus put a spurious shoulder beside the mountain, which the true conditional does not have.

Tile level. Here $y = H$. Because t is monotone in H , π_K reduces to four numbers per global interval J : all $H \in [a_0, b_1]$, $\min_J H \in [a_0, a_1]$ and $\max_J H \in [b_0, b_1]$.

- **Prolong:** a dynamic program over each interval, with four states for whether the low and high witnesses have been seen, minimising disagreement with the texture.
- **Smooth:**
 - $H(c) \pm 1$ moves, checked in $O(1)$ with per-interval witness counts;
 - swaps of two columns within an interval. These keep the interval’s set of heights, so every envelope is unchanged, and they move the witness role;
 - heat-bath over materials within the solid and air classes.

Fitting. Features are shared across interval positions:

- each interval’s (lo, hi) pair;
- the difference between adjacent intervals, and whether one is full while the other is empty;
- the interfaces of stacked intervals;
- each child interval against its parent interval.

Each child column’s likelihood is exact by the forward pass.

Results (128², $K = 16$, $h_{\text{top}} = 64$, $G = 8$).

Guarantee	0 unsupported cells and satisfaction 1.0 at every level, in every run (the previous pipeline repaired 28% of cells)
Clamps	always honoured, including out-of-distribution ones
Agreement	unclamped, $\tau = 0$: every statistic within $ z \leq 1$ of the exact p_G (height-histogram TV 0.06, promise TV 0.01)
Cost	promise pyramid ≈ 0.03 s, tile level ≈ 0.2 s

Three lessons follow from Section 5.

- **The target itself is flat.** At the tile set’s own parameters p_G is flat ground, so its pyramid carries almost no information. A mountain clamp then leaves $\hat{q}$ extrapolating: count tables are uninformed, and the linear predictor confidently spreads ridges, because it learned only “neighbours are alike”.
- **Coverage fixes seam snapping.** With a corpus of rolling hills, the share of surface columns lying exactly on a block seam falls from 0.38–0.48 to 0.10 (the corpus’s own is 0.06), and the flat caps disappear.
- **Remaining caps come from the inputs disagreeing.** The remaining flat-topped, stepped shapes come from the exemplar, which is built from rectangular fills, acting through τ and χ . With τ off, the output is rolling terrain.

7 The coordinate factors in texsyn

Prolongation U . For parent cell p and child offset $\delta \in \{0, 1\}^2$, $(US_{2h})(2p + \delta) = S_{2h}(p) + \lfloor (2\delta - 1)h/2 \rfloor \pmod{m}$: siblings are h apart, centred on the parent, and read the parent’s exemplar window.

Kernel k_h . Rounded uniform of half-width $h r_l$ ($l = L - \log_2 h$), from hashed noise ξ : $J_h(p) = \lfloor h r_l (2\xi_p - 1) + \frac{1}{2} \rfloor$. With the default $r = (1, 1, 0, \dots)$, k_h is a point mass below the top two levels.

Energy $\mathcal{E}_h$. With $F_h = G_{h/2} * F$ the blurred one-hot signature-and-socket features, P_h, μ_h the level’s PCA basis and mean, and $\Delta \in \{-2, \dots, 2\}^2$,

$$N_E^h(u) = P_h^\top ([F_h(u + h\Delta)]_\Delta - \mu_h), \quad N_S^h(p) = P_h^\top ([F_h(S(p + \Delta))]_\Delta - \mu_h),$$

$$\mathcal{E}_h(S) = \sum_p \|N_S^h(p) - N_E^h(S(p))\|^2 + \beta \mathbb{1}[h = 1] E_p(E[S]).$$

This is a nonparametric, coarse-grained patch energy: the exemplar is the model at every level, and the blur fixes which statistics a level matches.

Smoother R_h . Four checkerboard colours; each cell p of a colour picks from the 18-element multiset $\mathcal{C}(p) = \{S(p + \Delta) - h\Delta, C_{2h}(S(p + \Delta)) - h\Delta : \Delta \in \{-1, 0, 1\}^2\}$ with local energy $e_p(c) = w(c) \|N_S^h(p) - N_E^h(c)\|^2 + \beta \mathbb{1}[h = 1] \sum_d E_{\text{pair}}^d(E[c], E[S(p + d)])$, $w = 1$ for coherent candidates and κ for jumps, by Gumbel-max:

$$P(S(p) = c) \propto \text{mult}_{\mathcal{C}(p)}(c) e^{-e_p(c)/T}, \quad T = 0 \Rightarrow \text{ICM}(\text{argmin}).$$

Where R_h differs from a Gibbs kernel for g_h .

1. **Missing parent term.** e_p omits $-T \log k_h$.
2. **Missing blanket terms.** Only p 's own term is scored; the 24 other neighbourhoods containing $S(p)$ are ignored, and $N_S^h(p)$ uses p 's current value.
3. **State-dependent proposals.** $\mathcal{C}(p)$ depends on the neighbours and has no Metropolis–Hastings correction; multiplicities act as a log-count coherence bonus.
4. **Overlapping colours.** Same-colour cells are 2 apart, inside each other's 5×5 windows, so parallel updates read stale values (spacing ≥ 5 fixes it).

References

- [1] D. Koller and N. Friedman, *Probabilistic Graphical Models*, MIT Press, 2009. §4.6 (chain graphs), ch. 12 (ancestral and Gibbs sampling).
- [2] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006. §8.1.2, §11.3.
- [3] D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*, CUP, 2003. Ch. 29–30.
- [4] W. L. Briggs, V. E. Henson, S. F. McCormick, *A Multigrid Tutorial*, 2nd ed., SIAM, 2000.
- [5] S. Lefebvre and H. Hoppe, Parallel controllable texture synthesis, *SIGGRAPH* 2005.
- [6] R. Paget and I. D. Longstaff, Texture synthesis via a noncausal nonparametric multiscale Markov random field, *IEEE TIP* 7(6), 1998.

8 Connectivity in a block: local auxiliary fields

Code: `castlegen/blockfield.py`; test `notes/experiments/block_field.py`. Port semantics: `castlegen/blockconn.py`; the pipeline that produces the patterns: `notes/experiments/maze.py`.

Notation. All symbols of this section, defined once.

- $k = 16$: block side. $P = \{0, \dots, k-1\}^2$: the block's cells, written p, q . $N(p) \subseteq P$: the in-block 4-neighbours of p .
- $\mathcal{T}$: the tile set (signatures), $|\mathcal{T}| = 56$. $t_p \in \mathcal{T}$: the tile at p ; $x = (t_p)_{p \in P}$.
- $\text{node}(t) \in \{0, 1\}$: tile t is walkable (a room, hallway or courtyard tile).
- $\text{sock}_\sigma(t) \in \{0, 1\}$: side $\sigma \in \{N, E, S, W\}$ of t can join some walkable tile.
- $J(p, q) \in \{0, 1\}$ for $q \in N(p)$: p and q are joined, i.e. both walkable and their facing sockets connect (the tile set's join tables).
- Ports: the 8 half-sides of the block, $k/2$ edge cells each. The pattern $\ell \in \{0, 1..4, \text{OPEN}\}^8$ (from the promise, §below) gives each port *closed* (0), a *class* (1..4: openings joined outside the block), or *OPEN* (must have openings, assumes nothing outside).

- **Opening:** a walkable edge cell p with $\text{sock}_\sigma(t_p)$ on its outward side σ . $\text{exit}(p) \in \{0, 1\}$: p is an opening on a port with a class.
- **Fields:** $g_p \in \{0, \dots, G\} \cup \{\infty\}$, the number of gaps on the route p claims to an exit; $d_p \in \{0, \dots, D\}$, the route's step count (ignored when $g_p = \infty$). Here $G = 8$, $D = 96$. $\tilde{g} = g$ with $\tilde{\infty} = G + 1$.
- $s(p, q) = 1 - J(p, q)$: the gap cost of the step $p \rightarrow q$.
- $\lambda \geq 0$: the guidance weight, ramped $0 \rightarrow \lambda_{\max} = 8$. $\mu = \lambda$ (cost per gap, walkable cells), $\epsilon = 0.1 \mu$ (walls), $\delta = 0.05$ (per step), $L = 1000$ (per invalid cell).
- $E_{\text{tile}}(x)$: the tile energy at $T = 1$: pair tables over in-block pairs, closed ports facing wall, $-\log z(t)$ per tile, and the parent term $\beta \sum_p -\log \hat{p}(t_p | W_p)$, where W_p is the exemplar tiles within 2 of p 's coordinate in the parent window and $\hat{p}$ their smoothed frequencies.
- $c(p) = (x + 2y) \bmod 5$ for $p = (y, x)$: the update class.

Where the pattern comes from. The promise levels ($h = 64, 32, 16$) give each 16-block its open sides and a key; a side is *downhill* if open into a lower key. Both halves of a downhill side get class 1, both halves of another open side **OPEN**, and closed sides 0. Every block but the root has a downhill side.

Validity. Cell p is *valid* when

$$V_p = [g_p = \infty] \vee \text{exit}(p) \vee \exists q \in N(p) : g_q < \infty, d_q < d_p, g_q + s(p, q) \leq g_p.$$

Such a q is a *witness* of p . V_p reads only p and $N(p)$.

Energy.

$$E(x, g, d) = E_{\text{tile}}(x) + \lambda n_{\text{closed}}(x) + \lambda n_{\text{empty}}(x) + L \sum_p \mathbf{1}[\neg V_p] + \sum_p \left(\mu_p \tilde{g}_p + \delta d_p \mathbf{1}[g_p < \infty] \right),$$

with $\mu_p = \mu$ if $\text{node}(t_p)$, else ϵ ; n_{closed} the openings on closed ports (a sum over edge cells); n_{empty} the class or **OPEN** ports with no opening.

What zero means. If every cell is valid and $g_p = 0$ for every walkable p , then every walkable cell is joined inside the block to an exit: follow witnesses from p ; d strictly decreases, so the walk ends, and only at an exit; $g = 0$ forces every step to be a join ($s = 0$), since $g_q + s \leq g_p = 0$. The check is local: count invalid cells and walkable cells with $g > 0$.

The update. Every term of E but n_{empty} spans a cell and its neighbours, so two cells interact only within distance 2. Cells of one class $c(p)$ are at least 3 apart, so they share no term, and drawing them simultaneously from their single-site conditionals is exact.

Here $s_t(q, p)$ is the step cost with p 's tile set to t ; $\mu_t = \mu$ if $\text{node}(t)$, else ϵ . The last line of the energy charges L for each neighbour that relied on p as its witness and would lose it. n_{empty} couples the $k/2$ cells of a port; the update reads the port's other cells as they are, so it alone is not exact in parallel.

Algorithm 2 One sweep of the field sampler

```

1: for  $c = 0, \dots, 4$  do
2:   for all  $p$  with  $c(p) = c$  in parallel do
3:     for all  $q \in N(p)$  do
4:        $b_q \leftarrow V_q$  with  $p$  excluded as a witness
5:     end for
6:     for all candidates  $(t, g, d) \in \mathcal{T} \times (\{0..G\} \cup \{\infty\}) \times \{0..D\}$  do
7:        $e \leftarrow$  the terms of  $E$  that involve  $p$ , with  $(t_p, g_p, d_p) = (t, g, d)$ :
8:         tile pairs with  $N(p)$ ,  $-\log z(t)$ , parent term, port terms at  $p$ 
9:          $+ \mu_t \tilde{g} + \delta d \mathbb{1}[g < \infty]$ 
10:         $+ L \mathbb{1}[\neg V_p]$   $\triangleright$  own validity under  $(t, g, d)$ 
11:         $+ L \sum_{q \in N(p), \neg b_q} \mathbb{1}[\neg(g < \infty \wedge d < d_q \wedge g + s_t(q, p) \leq g_q)]$ 
12:      end for
13:       $(t_p, g_p, d_p) \leftarrow \arg \max_{(t, g, d)} (-e(t, g, d) + \text{Gumbel})$   $\triangleright$  exact draw at  $T = 1$ 
14:    end for
15: end for

```

Initialisation. x starts as the parent window. (g, d) start at the least valid values: a Dijkstra from the exits, ordered lexicographically on (g, d) , with step $q \rightarrow p$ adding $(s(p, q), 1)$; values past G or D become ∞ . The start is valid, and with L large the chain stays valid.

Why it guides. Walkable cells of an orphan component can only claim routes that cross gaps, so each pays μ per gap. Walls relay routes for $\epsilon \ll \mu$. Turning a wall next to a joined region into a joined walkable tile costs that cell nothing (it takes $g = 0$ from its witness), and lets the cells behind it lower g by one over the next sweeps. Paths grow toward orphans as a ratchet; the reward arrives a sweep or two later, through neighbour reads only. An invalid cell must cost a constant L : ramped with λ it is free early, and one invalid cell lets a whole orphan component claim $g = 0$.

Closed form over (g, d) . Enumerating the algorithm's candidates costs $|\mathcal{T}|(G+2)(D+1) \approx 54,000$ per site. The implementation draws the same conditional without enumerating. For a fixed tile t , the energy over the (g, d) grid ($g < \infty$) is

$$\mu_t g + \delta d + L \left(\mathbb{1}[\neg V_p(g, d)] + \sum_{q \text{ dependent}} \mathbb{1}[\neg \text{sat}_q(g, d)] \right),$$

where own validity is a union of quadrants, $V_p = \text{exit}(p) \vee \bigvee_q \{g \geq g_q + s_q, d \geq d_q + 1\}$ over neighbours with $g_q < \infty$, and each dependent (b_q false) is satisfied on a rectangle, $\text{sat}_q = \{g \leq g_q - s_q, d \leq d_q - 1\}$. Cutting the g axis at every $g_q + s_q$ and $g_q - s_q + 1$, and the d axis at every $d_q + 1$ and d_q , splits the grid into at most 9×9 rectangles of constant violation count v . A rectangle $[g_0, g_0 + n_g) \times [d_0, d_0 + n_d)$ has weight

$$e^{-Lv - \mu_t g_0 - \delta d_0} \frac{1 - e^{-\mu_t n_g}}{1 - e^{-\mu_t}} \frac{1 - e^{-\delta n_d}}{1 - e^{-\delta}}$$

(a sum of n terms when the rate is 0), and $g = \infty$ adds one entry, $e^{-\mu_t(G+1) - L n_{\text{dep}}}$. The sum depends on t only through its key: the gap bits s_q , exit and node, so each site computes it once per distinct key. A draw picks t by its tile terms plus its key's log-sum, then a rectangle (or ∞) by weight, then g and d independently within it by inverting truncated geometrics. Tested against

enumeration (`tests/test_blockfield.py`); 8 ms per sweep of a 16-block on one core, from 1.1 s enumerated.

Scope. Every class exit is a root, so the field certifies that each walkable cell reaches *some* class; required links between classes, and the root block (no class port: one component), need more (a field per group; a designated root cell).